Exp-8: Fine-Tuned LLM Deployment for Customer Support

AIM:

To fine-tune a language model for customer support and deploy it using Streamlit and evaluate its performance using BLEU and ROUGE metrics.

DATASET:

Customer Service / Bank FAQ Dataset

PROCEDURE:

Step 1: Import Libraries

from transformers import T5Tokenizer, T5ForConditionalGeneration, Trainer, TrainingArguments from datasets import Dataset import evaluate

Step 2: Load Dataset data

= {

"question": [

"How to reset password?",

"How to check balance?",

"How to contact support?"

],

"answer": [

"Click on forgot password and follow instructions.",

"Login to your account dashboard to view balance.",
"Contact support via email or phone."

] } dataset =

Dataset.from_dict(data)

Step 3: Preprocessing def

preprocess(example):

return {

"input_text": "Question: " + example["question"],

"target_text": example["answer"]

} dataset =

dataset.map(preprocess)

Step 4: Load Model tokenizer = T5Tokenizer.from_pretrained("t5-small") model =

T5ForConditionalGeneration.from_pretrained("t5-small")

Step 5: Tokenization def

tokenize(example):

inputs = tokenizer(example["input_text"],

padding="max_length", truncation=True) outputs = tokenizer(example["target_text"],

padding="max_length", truncation=True)

inputs["labels"] = outputs["input_ids"]

return inputs

tokenized_dataset = dataset.map(tokenize)

Step 6: Fine-Tuning training_args =
TrainingArguments(
output_dir="./results",
num_train_epochs=5,
per_device_train_batch_size=2
)
trainer = Trainer(model=model,
args=training_args,
train_dataset=tokenized_dataset
) trainer.train()

Step 7: Save Model model.save_pretrained("model")
tokenizer.save_pretrained("model")

Step 8: Deployment (Streamlit) import streamlit as st from transformers
import T5Tokenizer, T5ForConditionalGeneration tokenizer =
T5Tokenizer.from_pretrained("model") model =
T5ForConditionalGeneration.from_pretrained("model")
st.title("Customer Support Chatbot") query =
st.text_input("Enter your question:") if query:

input_text = "Question: " + query input_ids =
tokenizer.encode(input_text, return_tensors="pt") output =
model.generate(input_ids) answer = tokenizer.decode(output[0],
skip_special_tokens=True) st.write("Answer:", answer)

Step 9: Evaluation bleu = evaluate.load("bleu") rouge =
evaluate.load("rouge") predictions = ["Click on forgot

```
password and follow instructions.&quot;] references = [[&quot;Click on forgot
password and follow instructions.&quot;]] print(&quot;BLEU:&quot;,
bleu.compute(predictions=predictions, references=references))
print(&quot;ROUGE:&quot;, rouge.compute(predictions=predictions,
references=[&quot;Click on forgot password and follow instructions.&quot;]))
```

OUTPUT:

The chatbot successfully responds to customer queries through a deployed web interface.

RESULT:

The fine-tuned model generates accurate customer support responses and achieves good performance based on BLEU and ROUGE metrics.